Cloud Identity as Enterprise IdP — SAML App Integration Factory (SSO at Scale)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The problem . . . 4

The approach . . . 4

Why it is safe and governed . . . 4

The bottom line . . . 4

Solution Overview — Cloud Identity as Enterprise IdP: SAML App Integration Factory 5

What this is . . . 5

How it works . . . 5

Reference design . . . 6

Benefits . . . 6

Limitations & boundaries . . . 7

Deployment model . . . 7

Requirements 7

1. Super-admin (or delegated app-management admin) in Google Admin . . . 8

2. Governed Google Groups (or OUs) as the assignment mechanism . . . 8

3. Per-app service-provider (SP) admin access and SSO/provisioning capability . . . 8

4. App inventory and business owners . . . 9

5. Domain verification & IdP metadata/certificate handling . . . 9

6. Pilot cohort and test accounts . . . 9

7. Python 3.10+ with Google client libraries and ADC (for the tools) . . . 10

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

Implementation 12

Stage 0 — Assess (read-only) . . . 12

Stage 1 — Design the onboarding pattern . . . 13

Stage 2 — Configure a pilot set of apps . . . 14

Stage 3 — Pilot & validate . . . 15

Stage 4 — Expand the factory across the estate . . . 16

Testing & Validation 16

Test 1 — SP-initiated SSO . . . 16
Test 2 — IdP-initiated SSO . . . 17
Test 3 — Group-driven access (assignment lever) . . . 17
Test 4 — Provisioning: create . . . 17
Test 5 — Provisioning: deactivate (leaver) . . . 17
Test 6 — Attribute & NameID correctness . . . 18
Test 7 — Catalog & governance completeness . . . 18
Test 8 — Certificate-rotation dry run . . . 18
Pilot exit criteria (Gate 3 summary) . . . 18
Troubleshooting 18
SSO fails: "invalid assertion" / audience or Entity ID mismatch . . . 19
SSO fails: wrong user / "no account found" (NameID mismatch) . . . 19
Signature validation error after a certificate change . . . 19
An out-of-group user can still reach the app . . . 19
Guardrail check reports "assigned to ALL USERS" / "assigned to an OU" . . . 20
Auto-provisioning doesn't create accounts . . . 20
Removed user still has an active account (deprovisioning gap) . . . 20
Group changes don't take effect . . . 20
Tool error: "Directory API unavailable" / import error . . . 20
IdP-initiated SSO fails but SP-initiated works . . . 21
Best Practices 21
Standardize the pattern, then never deviate silently . . . 21
Assign by group, never by user . . . 21
Make provisioning follow assignment . . . 22
Treat the catalog as the system of record . . . 22
Manage certificates proactively . . . 22
Least privilege for admins and automation . . . 22
Pilot-first, with a real rollback . . . 22
Design security controls to your edition — and verify . . . 23
Keep security and compliance in the loop . . . 23
Deliverables Summary 23
Documentation . . . 23
The 5 Python tools (`gcloud` / Admin SDK — read-only or dry-run by default) . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

The outcome

Make **Google Cloud Identity / Workspace the single sign-on identity provider for the entire SaaS estate** — hundreds of third-party applications — through a **repeatable app-integration factory** rather than hundreds of one-off projects. New apps federate to Google SSO **fast, consistently, and auditably**, with access and account lifecycle driven from one directory lever.

The problem

In a Google-first enterprise, Google becomes the natural IdP for every SaaS app employees use. But onboarding each SAML/OIDC app individually — hand-crafting metadata, deciding provisioning, granting access per user — is slow, inconsistent, and hard to audit. The result is uneven security posture, orphaned SaaS accounts at offboarding, and no single view of who can reach what.

The approach

A standard **onboarding pattern** (`SSO Onboarding Pattern – SaaS – v1`) applied to every app, four building blocks deep:

- **One repeatable pattern** for SAML and OIDC alike — the same checklist every time.
- **Group-based assignment** — apps are turned on for governed directory groups, never per user, so joiner-mover-leaver automatically grants and revokes SaaS access.
- **Automatic provisioning** (Google auto-provisioning / SCIM) where the app supports it, scoped to the same group, so accounts in the app follow directory reality.
- **An app catalog** (`App Catalog – Federated SaaS – v1`) that records every integration's config, owner, provisioning method, and certificate expiry — the audit surface for access reviews.

Why it is safe and governed

The build follows a strict **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model: **pilot-first**, with explicit pass/fail gates and a documented per-app rollback. No app is turned on for its full audience until it passes validation against a pilot cohort — SSO works, provisioning creates *and* deactivates correctly, and access is provably group-driven. Five Python tools (Admin SDK / `gcloud`) automate discovery, validation, and guardrail checks; all are **read-only or dry-run by default**, with **no credentials or keys in code**. Regulatory and legal determinations remain with the organization's security and compliance functions; this solution supplies the technical method and the evidence.

The bottom line

Google as the single sign-on IdP for the whole SaaS estate — apps onboarded by a repeatable factory, provisioned and assigned by group — **fast, consistent, and auditable**, at a fraction of the cost of bespoke consulting.

Solution Overview — Cloud Identity as Enterprise IdP: SAML App Integration Factory

What this is

In a Google-first enterprise, **Cloud Identity / Google Workspace becomes the identity provider (IdP)** for the whole SaaS estate — hundreds of third-party applications that employees sign in to every day (Salesforce, ServiceNow, Slack, Workday, Atlassian, Zoom, Box, and long-tail line-of-business apps). Federating each of those apps to Google single sign-on (SSO) one app at a time — hand-crafting the SAML metadata, wiring up provisioning, deciding who gets access — is slow, inconsistent, and hard to audit.

This solution builds a **repeatable SAML/OIDC app-integration factory**: one standard onboarding pattern applied to every app, **group-based app assignment** driven from your directory, **automatic user provisioning** where the app supports it (SCIM / Google auto-provisioning connectors), and an **app catalog + runbook** that captures each integration's configuration and owner. The result is that new apps federate to Google SSO **fast, consistently, and securely**, and the whole program is governed from one place.

> ***Scope boundary.*** *This is **Google-as-IdP for third-party SaaS** — Google Cloud Identity / Workspace issuing SAML assertions and OIDC tokens to downstream apps. It is **distinct** from inbound Entra↔Google federation (making Microsoft Entra ID trust Google, or vice-versa, for workforce identity). Where an app's own admin console is involved, its internal click-paths are deferred to that vendor's documentation; the Google Admin side is authored at exact detail.*

How it works

The Google Admin console (**admin.google.com** → **Apps** → **Web and mobile apps**) is the control plane. For each app you either pick it from the **prebuilt SAML/OIDC app catalog** Google curates (thousands of apps with tested, semi-templated configuration) or add a **custom SAML app** when the app isn't in the catalog. Google issues signed SAML assertions (or OIDC ID tokens) to the app; the app trusts Google as its IdP.

Four building blocks make this a *factory* rather than a pile of one-off integrations:

1. **A standard onboarding pattern.** Every app follows the same ordered checklist — collect the app's

SP metadata, create the app in Google, exchange metadata/certificates, map attributes (NameID, email, groups), configure provisioning, assign by group, test, and record it in the catalog. The pattern is identical whether the app is SAML or OIDC; only the token format differs.

1. **Group-based app assignment.** Access to each app is turned **ON for specific Google Groups /

organizational units (OUs)**, never per-user. Group membership (sourced from your directory / HR / IGA system per Global Requirement 4) becomes the single lever that grants or revokes app access — the same lever your joiner-mover-leaver process already moves.

1. **Automatic provisioning where supported.** For apps that support it, Google **auto-provisioning**

(or the app's SCIM endpoint) creates, updates, and deactivates the user's account *in the app* when they're added to / removed from the assigned group — so access and account lifecycle stay in sync. Not

every app supports auto-provisioning; the catalog records which do.

1. **An app catalog + runbook.** A living inventory of every federated app: its type (SAML/OIDC),

NameID format, attribute mappings, assigned groups, provisioning method, business owner, and onboarding date. The catalog is the audit surface and the onboarding template for the next app.

Reference design

Named artifacts (reused verbatim throughout the guide, testing, and scripts):

- `SSO Onboarding Pattern – SaaS – v1` — the standard, documented onboarding checklist every app

follows (the "factory line").

- `App Catalog – Federated SaaS – v1` — the master inventory (a spreadsheet or IGA record) of every

app, its config, owner, and provisioning method.

- `Assignment Group – <App> – Users` — the Google Group whose membership grants access to a given

app (e.g. `Assignment Group – Salesforce – Users`). One per app (or per access role within an app).

- `Pilot Cohort – SSO Factory – Pilot` — a small OU/group of pilot users used to validate every new

integration before the app is turned on for its full audience.

- `Custom SAML App – <App> – v1` — the Google Admin app object for an app added as a custom SAML app

(not in the prebuilt catalog).

- `Attribute Map – <App> – v1` — the documented NameID + attribute/group mapping for an app.

Flow: **Directory / HR-IGA (source of truth)** → **Google Cloud Identity / Workspace (IdP)** → signed SAML assertion / OIDC token → **third-party SaaS app (SP)**, with the assigned Google Group driving both **who sees the app** and **who gets provisioned into it**.

Benefits

- **Speed:** a standard pattern turns app onboarding from a bespoke project into a repeatable checklist

a competent identity engineer can run in a scheduled change window.

- **Consistency:** every app is configured the same way (NameID format, attribute mapping, group

assignment, provisioning), so the estate is predictable and auditable.

- **One access lever:** group-based assignment ties app access to directory group membership, so

joiner-mover-leaver automatically grants and revokes SaaS access.

- **Lifecycle sync:** auto-provisioning keeps the app's user accounts in step with directory reality —

reducing orphaned accounts and license waste.

• **Governance:** the catalog is a single evidence surface for access reviews, SOX/ISO audits, and offboarding verification.

Limitations & boundaries

• **App SP configuration is vendor-specific.** Each app's own admin console (where you paste Google's IdP metadata and enable SCIM) differs; those steps are deferred to the app vendor's documentation. This solution standardizes the **Google side** and the **process**.

• **Not every app supports auto-provisioning.** SCIM / Google auto-provisioning availability varies by app and license tier; the catalog records the method (auto, JIT, or manual) per app.

• **Edition/feature dependence.** Some capabilities (e.g. context-aware access on app sign-in via **BeyondCorp Enterprise**, advanced session controls) depend on Workspace/Cloud Identity edition and may be preview or add-on features — **verify availability in your tenant** before designing them in.

• **Directory source of truth is a prerequisite, not built here.** Group membership must be governed upstream (Global Requirement 4). This solution consumes groups; it does not define your HR/IGA sync.

• **Regulatory determinations are deferred.** Which apps may hold regulated data, and any legal-hold or residency constraints, are decided by your security/compliance functions (Global Requirement 6).

Deployment model

This solution follows the house model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for any production change.

• **Assess** — inventory the SaaS estate, each app's SSO/provisioning readiness, and current access.

• **Design** — define `SSO Onboarding Pattern – SaaS – v1`, attribute-mapping standards, and naming.

• **Configure** — stand up the pattern and integrate a **pilot set** of representative apps.

• **Pilot** — validate SSO, provisioning, and group assignment against `Pilot Cohort – SSO Factory – Pilot` before turning any app on for its full audience.

• **Validate** — run the validation gates (SSO works, provisioning correct, assignment group-driven).

• **Expand** — run the factory across the estate, one app per change window, recording each in `App Catalog – Federated SaaS – v1`.

Requirements

These are the **solution-specific** prerequisites for building the SAML app-integration factory. They build **on top of** the shared **Global Requirements — Google Workspace & Gemini Solutions**

(`global-requirements-google.md`), which already cover: (1) a provisioned Cloud Identity / Workspace tenant with a super-admin, (2) the source Microsoft/Entra environment where relevant, (3) DNS control, (4) an **identity & provisioning source of truth**, (5) a pilot scope + change control, and (6) security/compliance stakeholders. Verify those first; the items below are what *this* solution adds.

Each requirement: **Description / Why required / How to verify / How to implement if missing.**

1. Super-admin (or delegated app-management admin) in Google Admin

• **Description:** An admin who can manage **Apps** → **Web and mobile apps** in admin.google.com — a super-admin, or a custom admin role with the **Services** → **Web and mobile apps** privilege (and **Users**/**Groups** read for assignment).

• **Why required:** Creating SAML/OIDC apps, exchanging metadata, configuring provisioning, and setting group-based assignment all require these privileges.

• **How to verify:** Sign in to **admin.google.com** → **Account** → **Admin roles**; confirm your account holds Super Admin or a role granting the *Web and mobile apps* privilege.

• **How to implement if missing:** Have a super-admin grant a least-privilege custom admin role (Admin roles → Create new role) scoped to app management, rather than handing out full super-admin.

2. Governed Google Groups (or OUs) as the assignment mechanism

• **Description:** Directory **Google Groups** (or OUs) that will drive per-app access — sourced/kept current from your identity source of truth (Global Requirement 4) via GCDS, SCIM, or your IGA. These become the `Assignment Group – <App> – Users` objects.

• **Why required:** The factory assigns apps **by group, never per user**; group membership is the single access lever. Ungoverned or manually maintained groups break the lifecycle guarantee.

• **How to verify:** In **admin.google.com** → **Directory** → **Groups**, confirm groups exist and their membership is populated by your sync (not hand-edited). Confirm the naming convention is agreed.

• **How to implement if missing:** Define the group model and provision it from the source of truth before assigning apps; if groups are hand-maintained today, put them under sync/IGA control first.

3. Per-app service-provider (SP) admin access and SSO/provisioning capability

• **Description:** Administrative access to **each target app's** own admin console, and confirmation the app supports **SAML 2.0 or OIDC** SSO (and, ideally, **SCIM / auto-provisioning**) on the licensed tier you hold.

• **Why required:** Federation is a two-sided trust: you configure Google (IdP) *and* the app (SP). Many

apps gate SSO/SCIM behind a specific ("enterprise"/"plus") license tier.

- **How to verify:** In each app, locate its SSO/SAML (or OIDC) settings and its SCIM/provisioning settings; confirm your license tier exposes them. Record findings in the readiness assessment (Tool 1).
- **How to implement if missing:** Obtain SP admin access from the app owner; if SSO/SCIM is gated by license, engage procurement to upgrade the app tier, or record the app as **manual-provisioning** in the catalog.

4. App inventory and business owners

- **Description:** A discoverable **inventory of the SaaS estate** — which apps exist, who owns each, and its current authentication method (local passwords, another IdP, or none) — feeding `App Catalog – Federated SaaS – v1`.
- **Why required:** You cannot onboard what you cannot see; the catalog is both the plan and the audit surface. Owners approve group assignment and provisioning scope.
- **How to verify:** Confirm a source for the inventory (an existing CMDB/CASB export, Workspace token audit, expense/SSO logs). Tool 1 produces an initial readiness view; owners are named per app.
- **How to implement if missing:** Assemble the inventory from CASB/CMDB/finance sources and OAuth token audit (**admin.google.com** → **Security** → **API controls** → **App access control**), then assign owners.

5. Domain verification & IdP metadata/certificate handling

- **Description:** Your primary domain **verified** in Google (from tenant setup) and a process to handle Google's **IdP metadata / SSO URL / entity ID / signing certificate**, including planning for **certificate rotation**.
- **Why required:** Apps trust Google's signing certificate; an expiring/rotated cert breaks every app that pinned it unless rotation is planned. Metadata must be exchanged accurately per app.
- **How to verify:** In **admin.google.com** → **Apps** → **Web and mobile apps**, open a SAML app's setup to view Google's **SSO URL, Entity ID, and Certificate** (download the IdP metadata). Confirm you can track each cert's expiry.
- **How to implement if missing:** Verify the domain (Global Requirement 3 covers DNS), and record the IdP metadata + certificate expiry in the catalog so rotation is scheduled, not discovered at outage.

6. Pilot cohort and test accounts

- **Description:** A small `Pilot Cohort – SSO Factory – Pilot` (OU or group) of real pilot users,

plus non-production **test accounts**, to validate each integration before full turn-on.

• **Why required:** The house model is pilot-first with pass/fail gates; every new app is proven against the pilot cohort before it is assigned to its full audience.

• **How to verify:** Confirm the pilot OU/group exists (Global Requirement 5) and contains members who can test each app; confirm test accounts exist in both Google and, where possible, the target app.

• **How to implement if missing:** Create the pilot group, populate it with willing pilot users across representative roles, and create test accounts before onboarding the first app.

7. Python 3.10+ with Google client libraries and ADC (for the tools)

• **Description:** A workstation/automation host with **Python 3.10+**, the **Google Cloud CLI (`gcloud`), and the** Google API Python client **(`pip install google-api-python-client google-auth`) authenticated via** Application Default Credentials (ADC) **or an admin OAuth flow with the** Admin SDK Directory API** enabled.

• **Why required:** The 5 included tools automate discovery, validation, and reporting through the Admin SDK (Directory/Groups) and `gcloud` — no hardcoded credentials or exported service-account keys.

• **How to verify:** Run `python --version` (≥3.10), `gcloud auth list`, and confirm ADC is set (`gcloud auth application-default print-access-token` returns a token). Confirm the **Admin SDK API** is enabled for the identity/automation project.

• **How to implement if missing:** Install Python 3.10+ and the Cloud SDK; run `gcloud auth application-default login` (or configure a workload-identity/service account with **domain-wide delegation** limited to the read-only Directory scopes the tools request). Never store a downloaded service-account key in the repo.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

- **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout,

and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

- **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model;

the pilot proves the design and change control protects production.

- **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

- **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

6. Security, compliance & governance stakeholders

- **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance**

stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

- **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to

enterprise policy and legal requirements, not just technical defaults.

- **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

- **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

This guide builds the SAML app-integration factory following the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every artifact is named with the convention `<Purpose> – <Scope> – <Stage>` and reused verbatim in later steps, testing, and the tools. **All production changes are pilot-first with a documented rollback and an explicit pass/fail gate.**

Google Admin console paths are current as of authoring; menu labels occasionally move — where a label differs, **verify the equivalent in your tenant**. Third-party app (SP) console steps are deferred to each vendor's documentation.

Stage 0 — Assess (read-only)

Goal: know the estate and each app's SSO/provisioning readiness before changing anything.

1. **Inventory the SaaS estate.** Assemble the app list from CASB/CMDB/finance sources plus the OAuth

token audit in **admin.google.com** → **Security** → **API controls** → **App access control** and (if licensed) the Workspace **investigation/login audit** logs. Record each app and a business owner.

1. **Run the readiness assessment (Tool 1).**

`python sso_readiness_assessment.py --apps apps.csv --out readiness.json` It reports, per app, the declared auth method, whether SSO/SCIM is claimed, and the owner — a read-only baseline. It never changes configuration.

1. **Seed the catalog.** Create `App Catalog – Federated SaaS – v1` (a spreadsheet or IGA record)

with columns: app, type (SAML/OIDC/unknown), NameID format, attribute map, assigned group, provisioning method (auto/JIT/manual), owner, IdP-cert expiry, onboarding date, status.

1. **Confirm the assignment groups exist and are governed** (Requirement 2): each app will map to an

`Assignment Group – <App> – Users` populated from your source of truth, not hand-edited.

Gate 0 (pass/fail): a seeded `App Catalog – Federated SaaS – v1`, each app owned, and the pilot cohort (`Pilot Cohort – SSO Factory – Pilot`) confirmed. **Fail** → do not proceed until the inventory and pilot exist.

Stage 1 — Design the onboarding pattern

Goal: one documented, repeatable pattern every app will follow.

1. **Author** `SSO Onboarding Pattern – SaaS – v1` — the standard checklist (this becomes the runbook):

2. Collect the app's **SP metadata** (ACS/Reply URL, Entity ID/audience, NameID format; for OIDC:

redirect URIs, required scopes).

1. Create the app in Google (prebuilt catalog entry **or** `Custom SAML App – <App> – v1`).

2. Exchange metadata/certificate both directions.

3. Define `Attribute Map – <App> – v1` (NameID → primary email by default; add `email`, given/

family name, and a **groups** claim only where the app consumes it).

1. Configure **provisioning** (Google auto-provisioning or the app's SCIM endpoint) where supported.

2. **Assign by group** — turn the app ON for `Assignment Group – <App> – Users` only.

3. Test against `Pilot Cohort – SSO Factory – Pilot`; record the result.

4. Record the integration in `App Catalog – Federated SaaS – v1`.

5. **Set standards** in the pattern: default **NameID = primary email (persistent)**; a fixed attribute-

naming convention; a rule that access is **group-only**; and a **certificate-rotation** entry for every app (track Google's IdP signing-cert expiry per app).

1. **Decide SAML vs OIDC per app** by what the SP supports (prefer the app's recommended method; both

are first-class in Google's app catalog). The pattern steps are identical; only token format differs.

Gate 1: `SSO Onboarding Pattern – SaaS – v1` is written, reviewed by security (Global Requirement 6), and names the attribute, NameID, group-only, and cert-rotation standards. **Fail** → finish the pattern before configuring apps.

Stage 2 — Configure a pilot set of apps

Goal: integrate a small **representative** pilot set (aim for 3–5 apps spanning: a prebuilt-catalog SAML app, a custom SAML app, an OIDC app, and one app with SCIM auto-provisioning) by running the pattern for each. Do all of this against the pilot only.

2a. Prebuilt-catalog SAML app (the common case)

1. In **admin.google.com** → **Apps** → **Web and mobile apps** → **Add app** → **Search for apps**, select the app.

Google pre-populates much of the SAML configuration.

1. On the **Google Identity Provider details** step, note the **SSO URL, Entity ID, and Certificate**

(download IdP metadata). Provide these to the app's SSO settings **in the app's own console** (vendor docs). Paste the app's **ACS URL and Entity ID** back into Google.

1. Configure `Attribute Map – <App> – v1`: set **Name ID = Basic Information > Primary email**,

format **EMAIL** (unless the app requires a different NameID — verify in the app's docs). Map any additional attributes the app requires.

1. **Do not turn the app ON for everyone.** On **User access**, turn it **ON only for**

`Assignment Group – <App> – Users` (or the pilot OU). Save.

2b. Custom SAML app (`Custom SAML App – <App> – v1`)

1. **Add app** → **Add custom SAML app.** Name it exactly `Custom SAML App – <App> – v1`.
2. Copy Google's **SSO URL, Entity ID, Certificate** (or download metadata) into the app's SSO config.
3. Enter the app's **ACS URL** and **Entity ID**; set **Name ID format** and **Name ID** per

`Attribute Map – <App> – v1`; add attribute mappings.

1. Set **User access** ON only for `Assignment Group – <App> – Users`. Save.

2c. OIDC app

1. Add the app from the catalog (or per the app's OIDC integration guide). Provide the app's **redirect

URIs** and the scopes it requires; consent to the OAuth client as directed.

1. Assign by group exactly as above (`Assignment Group – <App> – Users`).

2d. Provisioning (where supported)

1. If the app appears with **Auto-provisioning** in Google (Apps → the app → **Auto-provisioning**),

enable it, authorize the connection, and map attributes; **scope provisioning to** `Assignment Group – <App> – Users` so only assigned users are provisioned/deprovisioned.

1. If the app uses its **own SCIM endpoint**, generate a SCIM token in the app and configure the

provisioning connection per the app's docs; scope to the same assignment group.

1. Record the method (auto / SCIM / JIT / manual) in the catalog. **Availability of Google

auto-provisioning is per-app — verify it exists for your app before designing around it.**

Gate 2: each pilot app is created, attribute-mapped, assigned to its group only, and (where supported) provisioning-enabled — with **nothing turned on for the full audience yet**.

Stage 3 — Pilot & validate

Goal: prove each pilot integration against real pilot users before expanding.

1. Ensure pilot users are members of the relevant `Assignment Group – <App> – Users` (via your source

of truth). Add a test user to the pilot cohort where needed.

1. **SP-initiated and IdP-initiated SSO:** from the app's login page (SP-init) and from the user's

Google **apps** launcher (IdP-init, where the app supports it), sign in and confirm access.

1. **Provisioning check:** add a test user to the assignment group; confirm the account is created in

the app (auto/SCIM). Remove them; confirm deactivation. (See Testing for exact procedures.)

1. **Run the validation report (Tool 3).**

`python sso_validation_report.py --catalog catalog.csv --out validation.html` It reconciles the catalog against directory group membership and flags apps missing an assignment group, missing an owner, missing a provisioning method, or with an IdP cert nearing expiry.

1. **Run the guardrail check (Tool 4).**

`python assignment_guardrail_check.py --catalog catalog.csv` It flags policy violations: an app assigned to **All users / an OU instead of a group**, an app with **no group**, over-broad group assignment, or a NameID not matching the standard.

Gate 3 (the pilot gate): for every pilot app — SSO succeeds (SP- and IdP-initiated), provisioning create+deactivate works where enabled, assignment is **group-driven** (guardrail clean), and the validation report is green. **Fail** → fix and re-test; do **not** expand a failing app.

Rollback (documented, per app)

If an app fails validation or causes a production issue after turn-on:

1. In **Apps** → **Web and mobile apps** → **<app>** → **User access**, set the app **OFF** (or scope back to the

pilot group) — this immediately stops Google issuing assertions/tokens to that app.

1. If provisioning misbehaved, **disable auto-provisioning / SCIM** for the app so no further account

changes are pushed; reconcile any accounts already changed with the app owner.

1. Users fall back to the app's prior sign-in method (local login or previous IdP) **only if** it is

still enabled in the app — confirm the fallback exists **before** turn-on, and keep it until the app passes Gate 3. Record the rollback outcome in the catalog.

Stage 4 — Expand the factory across the estate

Goal: run the proven pattern across the remaining apps, one app per change window.

1. For each remaining app, execute `SSO Onboarding Pattern – SaaS – v1` end to end (Stages 2–3),

integrating and validating against the pilot cohort **before** turning the app on for its full `Assignment Group – <App> – Users`.

1. **Turn on for the full audience only after Gate 3 passes** for that app, inside an approved change

window (Global Requirement 5). Update `App Catalog – Federated SaaS – v1` (status → live, onboarding date, cert expiry).

1. **Batch by owner/risk:** low-risk, high-value apps first; regulated-data apps only after security/

compliance sign-off (Global Requirement 6).

1. **Operationalize the catalog with the config helper (Tool 5).**

`python catalog_config_helper.py --catalog catalog.csv --new-app "<App>" --type saml` It appends a correctly-shaped catalog row and emits the onboarding checklist for the new app so every integration is recorded consistently.

1. **Schedule certificate rotation** from the catalog's cert-expiry column; rotate Google's IdP signing

certificate and update each affected app's SP config before expiry (never after).

Gate 4 (steady state): every live app is in the catalog with an owner, an assignment group, a provisioning method, and a tracked cert expiry; the guardrail check runs clean across the estate; and new apps are onboarded only through `SSO Onboarding Pattern – SaaS – v1`.

Testing & Validation

Run these procedures at the pilot gate (Gate 3) for every app, and again on any change to an integration. Each test states the **expected pass/fail signal**. These map to the validation gates: **SSO works · provisioning correct · assignment is group-driven**. Use the included tools where noted; they are read-only / reporting and safe to run anytime.

Test 1 — SP-initiated SSO

• **Procedure:** As a pilot user in `Assignment Group – <App> – Users`, open the app's own login page

and choose "Sign in with Google / SSO". Complete the Google sign-in.

• **Pass:** the user lands authenticated in the app, mapped to the correct account (NameID = primary

email per `Attribute Map – <App> – v1`).

- **Fail:** SAML/OIDC error, "no account found", or wrong identity — check ACS/Entity ID, NameID format,

and attribute mapping.

Test 2 — IdP-initiated SSO

- **Procedure:** As the same pilot user, open the Google apps launcher (or

`https://myaccount.google.com/` → the app tile) and click the app.

- **Pass:** the app opens authenticated without a second login (where the app supports IdP-initiated).
- **Fail:** the app rejects the assertion (some apps are SP-init only — record that in the catalog, not

a defect) or errors on audience/Entity ID mismatch.

Test 3 — Group-driven access (assignment lever)

- **Procedure:** Confirm the app is turned **ON only for** `Assignment Group – <App> – Users` (Apps →

Web and mobile apps → <app> → User access). Then take a pilot user who is **not** in that group and attempt SSO.

- **Pass:** in-group user gets access; out-of-group user is **denied** by Google before reaching the app.
- **Fail:** an out-of-group user reaches the app → assignment is not group-scoped (likely ON for an OU or

All users). Remediate and re-run Tool 4.

- **Automated check:** `python assignment_guardrail_check.py --catalog catalog.csv` — **pass** = no

findings; **fail** = any app flagged as All-users/OU-assigned, no-group, or over-broad.

Test 4 — Provisioning: create

- **Procedure (apps with auto-provisioning/SCIM only):** Add a **test user** to

`Assignment Group – <App> – Users`. Wait for the provisioning cycle.

- **Pass:** the user account is **created/activated in the app** with the mapped attributes; the app

owner can see the new account.

- **Fail:** no account appears within the expected window → check the provisioning connection, scope

(must be the assignment group), and attribute mapping.

Test 5 — Provisioning: deactivate (leaver)

- **Procedure:** Remove the same test user from `Assignment Group – <App> – Users`.
- **Pass:** the app account is **deactivated/suspended** (or deleted, per your policy) on the next cycle;

the user can no longer sign in.

• **Fail:** the account remains active → orphaned-account risk. Verify deprovisioning is enabled and scoped to the group. This is the highest-value offboarding test — do not skip it.

Test 6 — Attribute & NameID correctness

• **Procedure:** Inspect the assertion/token the app received (app-side SSO logs, or a SAML tracer/JWT decode of a test login) and compare to `Attribute Map – <App> – v1`.

• **Pass:** NameID and each mapped attribute (email, name, groups where used) match the standard.

• **Fail:** missing/mis-mapped attribute → fix the mapping in Google and re-test.

Test 7 — Catalog & governance completeness

• **Procedure:** `python sso_validation_report.py --catalog catalog.csv --out validation.html` and open the report.

• **Pass:** every app row has a type, an assignment group, an owner, a provisioning method, and an IdP-cert expiry that is **not** within the warning window; no app is missing from the catalog.

• **Fail:** any red row (missing owner/group/method, or cert expiring soon) → remediate before that app is considered "done".

Test 8 — Certificate-rotation dry run

• **Procedure:** From the catalog's cert-expiry column, pick an app and rehearse rotation: rotate/stage a new Google IdP signing certificate and update the app's SP config in a **test** window; sign in.

• **Pass:** SSO succeeds on the new certificate; the app trusts the rotated cert.

• **Fail:** SSO breaks on rotation → the app pinned the old cert; schedule a maintenance window and update the SP before the live cert expires.

Pilot exit criteria (Gate 3 summary)

An app **passes the pilot** only when Tests 1–7 pass (Test 8 rehearsed at least once for the estate): SP- and IdP-initiated SSO succeed, access is denied outside the group, provisioning creates **and** deactivates correctly (where enabled), attributes match the standard, and the validation report and guardrail check are both clean. Only then is the app turned on for its full audience (Stage 4).

Troubleshooting

Common failure modes when running the SAML app-integration factory, and how to resolve them. Google Admin labels occasionally move — where a path differs, **verify the equivalent in your tenant**. Third-party

app (SP) console specifics are deferred to each vendor's documentation.

SSO fails: "invalid assertion" / audience or Entity ID mismatch

- **Symptom:** the app rejects Google's SAML assertion; error mentions audience, Entity ID, or issuer.
- **Cause:** the SP **Entity ID / audience** entered in Google doesn't exactly match what the app

expects, or Google's **Entity ID** wasn't entered on the app side.

- **Resolve:** in **Apps** → **Web and mobile apps** → **<app>** → **Service provider details**, confirm the app's

ACS URL and Entity ID are exact (no trailing slash, correct scheme). Re-copy Google's SSO URL / Entity ID / metadata into the app. Re-run **Test 1**.

SSO fails: wrong user / "no account found" (NameID mismatch)

- **Symptom:** SSO completes at Google but the app can't match the user, or logs in the wrong account.
- **Cause:** the **NameID** doesn't match the identifier the app keys on (often the app wants email but

received something else), or the NameID **format** is wrong.

- **Resolve:** set **Name ID = Basic Information > Primary email**, format **EMAIL**, per

`Attribute Map – <App> – v1` — unless the app documents a different NameID. Confirm the user's app account uses the same email. Re-run **Test 6**, then **Test 1**.

Signature validation error after a certificate change

- **Symptom:** every user of an app suddenly fails SSO; app logs a signature/cert error.
- **Cause:** Google's **IdP signing certificate rotated or expired** and the app pinned the old cert.
- **Resolve:** download Google's current certificate/metadata (**<app> → Google Identity Provider

details**) and update the app's SP config. Then** prevent recurrence:** record the new expiry in App `Catalog – Federated SaaS – v1` and schedule rotation before expiry (Test 8). Prefer metadata-URL exchange where the app supports it so rotation propagates automatically.

An out-of-group user can still reach the app

- **Symptom:** someone not in `Assignment Group – <App> – Users` gets into the app.
- **Cause:** the app is turned **ON for an OU or All users**, not scoped to the group — or the app also

allows **local login** that bypasses SSO.

- **Resolve:** in **<app>** → **User access**, set ON **only** for `Assignment Group – <App> – Users`; turn

off inheritance from broader OUs. If the app allows non-SSO login, enforce SSO-only in the app (vendor docs). Re-run **Test 3** and the guardrail check (`python assignment_guardrail_check.py --catalog catalog.csv`).

Guardrail check reports "assigned to ALL USERS" / "assigned to an OU"

- **Symptom:** `assignment_guardrail_check.py` exits non-zero with an assignment violation.
- **Cause:** the catalog's `assignment_target` is `all-users` or `ou`, violating the group-only standard.
- **Resolve:** create/confirm `Assignment Group – <App> – Users`, re-scope the app's **User access** to

that group in Google Admin, update the catalog row (`assignment_target=group`, set `assignment_group`), and re-run the check until clean.

Auto-provisioning doesn't create accounts

- **Symptom:** a user added to the assignment group never gets an account in the app.
- **Cause:** auto-provisioning/SCIM not enabled, **not scoped to the assignment group**, an expired SCIM

token, or the app doesn't support Google auto-provisioning on your tier.

- **Resolve:** in **<app>** → **Auto-provisioning**, confirm it's enabled and **scoped to

`Assignment Group – <App> – Users`**; re-authorize the connection / regenerate the app's SCIM token if needed; verify attribute mapping. If the app has no provisioning connector, record method** manual **in the catalog and handle onboarding manually. Re-run** Test 4**.

Removed user still has an active account (deprovisioning gap)

- **Symptom:** a user removed from the assignment group can still sign in to the app.
- **Cause: deprovisioning disabled**, provisioning scoped to the wrong group, or the app is

manual/JIT with no automated leaver action.

- **Resolve:** enable deactivation/suspension in the provisioning connection scoped to the assignment

group; for manual/JIT apps, ensure the app is on the **leaver offboarding checklist**. Re-run **Test 5** — this is the highest-value offboarding control; do not leave it failing.

Group changes don't take effect

- **Symptom:** membership changes in your source of truth aren't reflected in access or provisioning.
- **Cause:** the group is **hand-edited** rather than governed by the sync/IGA, or the sync (GCDS/SCIM)

hasn't run.

- **Resolve:** confirm the group is populated by the source of truth (Requirement 2), not manually;

verify the sync cycle completed. Re-run `sso_readiness_assessment.py --check-groups` to confirm the group exists and is populated.

Tool error: "Directory API unavailable" / import error

- **Symptom:** a tool prints that the Admin SDK client or credentials are unavailable.

• **Cause:** `google-api-python-client` / `google-auth` not installed, ADC not set, or the **Admin SDK

API** not enabled / insufficient scope.

• **Resolve:** `pip install google-api-python-client google-auth`; run

`gcloud auth application-default login`; enable the **Admin SDK API** on the automation project; ensure the account has the read-only Directory scopes (or a service account with **domain-wide delegation** limited to those read-only scopes). The tools run their file/inventory logic without the API, and only the `--check-groups` / `--check-membership` steps need it.

IdP-initiated SSO fails but SP-initiated works

• **Symptom:** the app works from its own login page but errors from the Google apps launcher.

• **Cause:** the app is **SP-initiated only** (a legitimate app design), or the IdP-init relay/audience

is misconfigured.

• **Resolve:** confirm whether the app supports IdP-initiated SSO (vendor docs). If it doesn't, record

"SP-init only" in the catalog — this is a documented characteristic, not a defect (**Test 2**).

Best Practices

Durable practices for running the SAML app-integration factory as a program, not a series of one-off projects.

Standardize the pattern, then never deviate silently

• Onboard **every** app through `SSO Onboarding Pattern – SaaS – v1`. If an app genuinely needs a

deviation (unusual NameID, SP-init only, no provisioning), record the deviation **in the catalog** — the exception is documented, not invisible.

• Keep the pattern versioned. When you improve it (`v2`), migrate apps deliberately, not by drift.

Assign by group, never by user

• Turn apps ON for `Assignment Group – <App> – Users` only. **Never** assign to "All users" or a broad

OU as a shortcut — that severs the link between directory lifecycle and app access and is the #1 finding the guardrail check catches.

• Keep assignment groups **governed by the source of truth** (Global Requirement 4). A hand-edited group

is an ungoverned access grant.

• One group per app (or per access role within an app) keeps access reviews legible.

Make provisioning follow assignment

• Scope auto-provisioning / SCIM to the **same** `Assignment Group – <App> – Users` that grants access,

so "in the group" means both *can sign in* and *has an account*. Divergence creates orphaned accounts.

• Always test **deprovisioning** (Test 5), not just creation. Offboarding is where orphaned SaaS accounts

and license waste hide.

• For apps without provisioning, record **manual/JIT** in the catalog and put their offboarding on the

leaver checklist explicitly.

Treat the catalog as the system of record

• `App Catalog – Federated SaaS – v1` is the audit surface: type, attribute map, group, provisioning

method, owner, cert expiry, status. Update it as part of "done" for every app — a live app not in the catalog does not exist for governance.

• Use the catalog to drive access reviews and offboarding verification, and to answer "which apps does a

leaver still have?" from group membership.

Manage certificates proactively

• Record Google's **IdP signing-certificate expiry per app** and rotate **before** expiry, in a planned

window — an expired cert breaks every app that pinned it. Rehearse rotation (Test 8) at least once.

• Prefer metadata-URL exchange over pasted certs where the app supports it, so rotation propagates

automatically.

Least privilege for admins and automation

• Grant app-management via a **custom admin role** (Web and mobile apps privilege) rather than blanket

super-admin.

• Run the tools with **read-only ADC** or a service account with **domain-wide delegation limited to the

read-only Directory scopes **they need.** Never** commit a service-account key; never hardcode secrets. Keep write actions (the config helper's file writes) `--dry-run` by default.

Pilot-first, with a real rollback

• Every app is validated against `Pilot Cohort – SSO Factory – Pilot` before full turn-on. Keep the

app's prior sign-in method available until it passes Gate 3, so rollback is real, not theoretical.

- Turn apps on for their full audience only inside an approved change window.

Design security controls to your edition — and verify

- Session length, re-authentication, and **context-aware access** (BeyondCorp Enterprise) on app sign-in

depend on your Cloud Identity / Workspace edition and may be add-on or preview features. **Verify availability in your tenant** before you commit to them in the pattern; don't assume a control exists.

Keep security and compliance in the loop

- Have security review `SSO Onboarding Pattern – SaaS – v1` and sign off on regulated-data apps before

onboarding (Global Requirement 6). Legal/records determinations stay with those functions; you supply the technical method and the catalog as evidence.

Deliverables Summary

Everything included in this solution, on one page.

Documentation

- **Solution Overview** — Google Cloud Identity / Workspace as the enterprise IdP: SAML vs OIDC, the app

catalog, auto-provisioning, group-based assignment, the reference design, and the deployment model.

- **Requirements Guide** — the solution-specific prerequisites (app-management admin, governed

assignment groups, per-app SP access, app inventory/owners, IdP metadata/cert handling, pilot cohort, Python/ADC tooling), each with how to *verify* and *implement*, building on the shared Google category requirements.

- **Implementation Guide** — the ordered build following **Assess → Design → Configure → Pilot →

Validate → Expand**: define `SSO Onboarding Pattern – SaaS – v1`, integrate a pilot set of SAML/OIDC apps, wire auto-provisioning + group-based assignment, publish `App Catalog – Federated SaaS – v1`, then scale the factory — with pass/fail gates and a documented rollback.

- **Testing & Validation** — eight procedures (SP/IdP-initiated SSO, group-driven access, provisioning

create + deactivate, attribute/NameID correctness, catalog completeness, cert-rotation dry run) with explicit pass/fail signals and the pilot exit criteria.

- **Troubleshooting** — common failure modes (assertion errors, NameID mismatch, provisioning not firing,

out-of-group access, cert expiry) and their resolutions.

- **Best Practices** — group-only assignment, provisioning-follows-assignment, the catalog as system of

record, proactive certificate rotation, least-privilege admin/automation, and pilot-first rollout.

• **Executive Summary** — the outcome, approach, and why it is safe and governed, for decision-makers.

The 5 Python tools (`gcloud` / Admin SDK — read-only or dry-run by default)

1. `sso_readiness_assessment.py` — *Assessment / discovery (read-only).* Reads an app inventory and,

via the Admin SDK Directory API, reports each app's declared auth method, SSO/SCIM readiness, and owner — the baseline for the catalog. Changes nothing.

1. `saml_app_onboarding_helper.py` — *Configuration / provisioning helper (dry-run default).* Given

an app's SP details, generates the standardized onboarding checklist, the `Attribute Map – <App> – v1` template, and the exact Google-Admin steps to run; writes artifacts only with `--apply`.

1. `sso_validation_report.py` — *Validation / report generator (read-only).* Reconciles

`App Catalog – Federated SaaS – v1` against directory group membership and emits an HTML/JSON report flagging apps missing an owner, assignment group, provisioning method, or with a cert nearing expiry.

1. `assignment_guardrail_check.py` — *Policy / guardrail check (read-only).* Flags policy violations

— apps assigned to All-users/an OU instead of a group, apps with no assignment group, over-broad assignment, or NameID not matching the standard. Non-zero exit on findings for CI use.

1. `catalog_config_helper.py` — *Scope / config helper (dry-run default).* Appends a correctly-shaped

row to the catalog for a new app and emits its onboarding checklist, keeping every integration recorded consistently; writes only with `--apply`.

Safety: every tool is read-only or `--dry-run` unless `--apply` is passed; none contain hardcoded credentials or service-account keys; all authenticate via `gcloud` / Application Default Credentials and request the least Directory scopes needed.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

assignment_guardrail_check.py

catalog_config_helper.py

saml_app_onboarding_helper.py

sso_readiness_assessment.py

sso_validation_report.py